

Revisão local de segurança — Sistema de Contratações Públicas

Data: 17 de agosto de 2026

Escopo: revisão estática do código-fonte e da configuração presente no repositório local.

Fora do escopo: domínio público, Cloudflare, portas, SSH, sistema operacional, banco em produção, testes de intrusão e validação jurídica formal.

Resumo executivo

O sistema já possui uma base positiva: senhas com Argon2, tokens de sessão aleatórios armazenados somente como hash, consultas SQL parametrizadas, cookies `HttpOnly / SameSite=Lax`, CSP, desativação da documentação automática e autorização por perfil e secretaria no servidor.

Ainda assim, **não é recomendável considerar a aplicação pronta para exposição pública permanente** antes de corrigir os itens críticos e altos abaixo. Os maiores riscos são: tomada da primeira conta administrativa durante uma instalação vazia, ausência de limitação de tentativas de login, processamento sem limite de arquivos enviados, injeção de fórmulas nas planilhas produzidas e uso de túnel público temporário sem uma camada explícita de acesso institucional.

ID	Severidade	Achado	Prioridade
SEG-01	Crítica	Configuração inicial permite reivindicar a primeira conta administrativa	Imediata
SEG-02	Alta	Login sem rate limit, bloqueio progressivo ou proteção contra força bruta	Imediata
SEG-03	Alta	Upload é lido integralmente, sem limite de bytes/páginas/complexidade	Imediata
SEG-04	Alta	Textos controlados pelo usuário podem virar fórmulas em XLSX	Imediata
SEG-05	Alta	Script publica um Quick Tunnel acessível pela internet	Imediata
SEG-06	Média	Corpos JSON e campos de texto não possuem limites estruturais	Curto prazo
SEG-07	Média	Troca de senha não revoga sessões existentes	Curto prazo
SEG-08	Média	Título de contratação é montado com <code>innerHTML</code>	Curto prazo
SEG-09	Média	Auditoria não cobre leituras, downloads e eventos suficientes	Curto prazo
SEG-10	Média	Dependências têm apenas versões mínimas e não há lockfile	Curto prazo
SEG-11	Média	Controles operacionais de dados, backup e retenção não estão definidos	Antes da produção
SEG-12	Baixa	Não há validação explícita de hosts/proxy confiável	Planejado

Achados detalhados

SEG-01 — Tomada da primeira conta administrativa

Evidência: `server.py:188-212`. O endpoint público `/api/auth/status` informa que a instalação está vazia e `/api/auth/setup` cria um administrador quando `total_usuarios() == 0`.

Impacto: se uma instância vazia for publicada antes da configuração, qualquer pessoa que encontre a URL pode tentar criar a primeira conta administrativa. A verificação e a inserção também não formam uma única operação atômica.

Recomendação: retirar o setup da interface pública. Exigir token de bootstrap de uso único fornecido por variável de ambiente, ou criar o primeiro administrador por comando executado localmente. Fazer a garantia de “primeiro usuário” dentro de transação e desativar permanentemente o bootstrap após o uso.

SEG-02 — Ausência de proteção contra força bruta

Evidência: `server.py:220-233` e `auth.py:68-78`. Toda tentativa é validada diretamente por Argon2; não existe limite por IP/login, atraso progressivo, bloqueio temporário ou segundo fator.

Impacto: permite tentativas repetidas de descoberta de senha e também consumo elevado de CPU, pois Argon2 é intencionalmente custoso.

Recomendação: aplicar rate limit no proxy e na aplicação, com chave combinando IP e login; atraso progressivo; alerta/bloqueio temporário; MFA para administradores; mensagens genéricas e monitoramento de falhas.

SEG-03 — Upload sem limites

Evidência: `server.py:474-482` chama `await arquivo.read()` sem limite. `requisicao.py:46-84` abre XLSX e `requisicao.py:99-158` processa todas as páginas/tabelas do PDF. A extensão é validada, mas o conteúdo pode ser aceito por assinatura PDF mesmo com outro nome.

Impacto: arquivos grandes, PDFs complexos e arquivos XLSX compactados de forma hostil podem esgotar memória, CPU ou espaço temporário e derrubar o serviço.

Recomendação: limitar o corpo no proxy e na aplicação; fazer leitura incremental com interrupção; definir limites de tamanho, páginas, dimensões, células e tempo; validar MIME e assinatura; processar em trabalhador isolado com timeout; manter bibliotecas de parsing atualizadas.

SEG-04 — Injeção de fórmulas em planilhas

Evidência: `requisicao.py:184-206`. Fornecedor, endereço, cidade, identificação e descrições são gravados diretamente em células. Strings iniciadas por `=`, `+`, `-` ou `@` podem ser interpretadas pelo Excel como fórmulas.

Impacto: ao abrir a requisição, uma fórmula maliciosa pode manipular o documento, induzir acesso a recursos externos ou explorar recursos perigosos disponíveis no cliente de planilhas.

Recomendação: neutralizar texto não confiável antes de gravá-lo em XLSX, prefixando apóstrofo quando começar por caracteres de fórmula; distinguir células de fórmula criadas pelo próprio sistema das células de texto; adicionar testes específicos.

SEG-05 — Publicação por Quick Tunnel

Evidência: `publicar_cloudflare.bat:29-33` executa `cloudflared tunnel --url ...` e divulga um endereço `trycloudflare.com`. Não há no repositório política de Cloudflare Access, allowlist, identidade institucional ou túnel nomeado.

Impacto: o endereço é público e a autenticação da própria aplicação vira a única barreira. A URL temporária dificulta governança, inventário, regras persistentes, logs e resposta a incidentes.

Recomendação: não usar Quick Tunnel em produção. Empregar hospedagem administrada ou túnel nomeado, domínio institucional, TLS obrigatório, Cloudflare Access/SSO ou VPN, allowlist quando viável, WAF/rate limiting e logs centralizados. Manter Uvicorn somente em interface privada/loopback atrás do proxy.

SEG-06 — Entradas sem limites estruturais

Evidência: modelos em `server.py:47-96` usam `dict[str, Any]` e strings sem `Field(max_length=...)`. As rotas de geração aceitam documentos completos e produzem DOCX/XLSX em memória.

Impacto: usuários autenticados ou contas comprometidas podem enviar JSON muito grande, campos enormes e estruturas inesperadas, causando consumo de memória/CPU e registros excessivos no banco.

Recomendação: criar modelos Pydantic específicos por documento, proibir campos extras, limitar comprimentos e quantidades, validar números e datas e impor limite global de corpo.

SEG-07 — Sessões permanecem válidas após troca de senha

Evidência: `auth.py:80-87` altera a senha, mas não remove sessões. `database.py:302-332` oferece criação/remoção individual, sem revogação de todas as sessões do usuário.

Impacto: uma sessão roubada continua válida por até oito horas mesmo depois de a vítima trocar a senha. A desativação do usuário, por outro lado, é verificada a cada requisição e bloqueia corretamente a sessão.

Recomendação: revogar todas as sessões ao alterar senha, login, perfil ou secretaria; permitir “sair de todos os dispositivos”; registrar criação e revogação; considerar expiração ociosa além da expiração absoluta.

SEG-08 — Construção insegura de opções no front-end

Evidência: `frontend/app.js:645-654` concatena `item.titulo`, originado do banco, em `innerHTML`.

Impacto: um título especialmente construído pode alterar o DOM. A CSP atual bloqueia scripts inline e reduz o impacto, mas não torna `innerHTML` seguro nem protege contra todas as formas de injeção de marcação.

Recomendação: criar elementos `option` com `document.createElement`, definir `value` e `textContent`; reservar `innerHTML` apenas para fragmentos totalmente constantes.

SEG-09 — Auditoria insuficiente para processo administrativo

Evidência: `database.py:334-357` possui infraestrutura de auditoria, usada em login, usuários, criação de contratação e versões. Não há registro consistente de consulta, download, geração, importação, emissão de relatório, alteração de senha, IP, agente do navegador ou resultado de autorização negada.

Impacto: investigação de incidente e comprovação de autoria/cadeia de ações ficam incompletas.

Recomendação: definir matriz formal de eventos; registrar ator, instante UTC, ação, recurso, resultado, IP confiável do proxy, agente e identificador de correlação, sem gravar senhas ou conteúdo sensível desnecessário. Proteger logs contra alteração e definir retenção.

SEG-10 — Dependências não reproduzíveis

Evidência: `requirements.txt` usa somente limites mínimos (`>=`) e não existe lockfile. `pip check` não encontrou incompatibilidades no ambiente atual, mas isso não é uma análise de vulnerabilidades conhecidas.

Impacto: instalações em momentos diferentes recebem versões distintas; uma atualização indireta pode quebrar o sistema ou introduzir risco de cadeia de suprimentos.

Recomendação: gerar lockfile com hashes, automatizar atualização controlada, executar auditoria de CVEs em CI e criar política de correção. Registrar SBOM para implantação institucional.

SEG-11 — Governança de dados não definida no código/documentação

Evidência: o banco armazena usuários, documentos, fornecedores, placas, valores e auditoria; `.gitignore` exclui o SQLite local, o que é positivo. Não foram encontrados no repositório documentos sobre classificação, retenção, descarte, backup, restauração, criptografia em repouso ou resposta a incidentes.

Impacto: perda, retenção excessiva ou acesso indevido a dados administrativos e eventualmente pessoais. A adequação depende do processo real da prefeitura, não apenas do código.

Recomendação: inventariar dados e bases legais com encarregado/jurídico; aplicar minimização e perfis; definir retenção por classe documental; backups criptografados e testados; plano de continuidade e incidente; termos de responsabilidade; registro formal dos administradores. Esta observação não substitui parecer jurídico ou arquivístico.

SEG-12 — Host e cabeçalhos de proxy

Evidência: não há `TrustedHostMiddleware`; `server.py:121-128` confia em `X-Forwarded-Proto` para HSTS/cookie seguro. O servidor local é iniciado em loopback, o que reduz a superfície quando o procedimento é seguido.

Impacto: implantação incorreta ou exposição direta pode aceitar hosts arbitrários e cabeçalhos encaminhados não confiáveis.

Recomendação: limitar hosts aceitos, definir proxies confiáveis na execução, configurar HTTPS no proxy e manter `COOKIE_SECURE=true` em produção. Não usar cabeçalho fornecido pelo cliente como única garantia de transporte seguro.

Controles positivos observados

- Hash de senha Argon2 via `pwdlib` (`auth.py:37` , `auth.py:64`).
- Token de sessão aleatório e persistência somente do SHA-256 (`auth.py:39` , `auth.py:89-97`).
- Sessão expira e usuário inativo é rejeitado (`database.py:317-328`).
- Cookie `HttpOnly` , `SameSite=Lax` e suporte a `Secure` (`server.py:133-143`).
- Consultas com parâmetros na maior parte do repositório; não foi encontrada concatenação de dados externos em SQL.

- CSP, proteção contra framing, `nosniff`, política de referenciador e HSTS sob HTTPS (`server.py:113-128`).
- Swagger/OpenAPI desativados (`server.py:40-41`).
- Separação de administrador/editor e autorização de secretaria aplicada no servidor (`server.py:145-176` , `server.py:289-355` , `server.py:485-527`).
- Senhas e banco local excluídos do Git; busca no histórico disponível não encontrou segredo real conhecido.
- Não foram encontrados `eval` , `pickle` , `shell=True` ou execução de comandos a partir de entrada HTTP.

Plano recomendado

Antes de qualquer nova publicação pública

1. Fechar o bootstrap administrativo (SEG-01).
2. Adicionar rate limit e política de login (SEG-02).
3. Limitar e isolar uploads (SEG-03).
4. Neutralizar fórmulas em XLSX (SEG-04).
5. Substituir Quick Tunnel por acesso institucional controlado (SEG-05).
6. Ativar `COOKIE_SECURE=true` , HTTPS obrigatório e hosts confiáveis.

Antes do uso interno oficial

1. Modelar e limitar todos os payloads (SEG-06).
2. Revogar sessões em eventos sensíveis (SEG-07).
3. Remover o `innerHTML` com dados persistidos (SEG-08).
4. Completar auditoria e retenção (SEG-09/SEG-11).
5. Fixar dependências, auditar CVEs e automatizar testes (SEG-10).
6. Executar testes de integração de autorização para cada rota e perfil.
7. Documentar backup/restauração e realizar ensaio de recuperação.

Validações realizadas

- Leitura estática dos arquivos Python, JavaScript, HTML, scripts e configuração.
- Busca por execução dinâmica, comandos de sistema, desserialização insegura, URLs, uploads e sinks de DOM.
- Busca por segredos no conteúdo e no único commit disponível: nenhum segredo real identificado.
- `python -m pip check` : nenhuma dependência quebrada no ambiente atual.
- Suíte local: 32 testes aprovados nesta revisão.

Limitações

Esta revisão não confirma a segurança da infraestrutura real, regras Cloudflare, firewall, TLS, permissões do sistema operacional, configuração MySQL, backups, logs ou segredos de produção. Também não incluiu scanner de CVEs, SAST dedicado, DAST, fuzzing nem pentest. Uma revisão externa autorizada deve ocorrer depois da correção dos achados e da definição da arquitetura de implantação.